Project ④ 비동기 프로그래밍을 통한 실시간 PLC 제어 평가 (학생용)

화요일 완성본(PICKNPLACE_FULL, A~D 구현)에 반복 운전을 더한다. 코드는 AI 에이전트가 작성하고, 학생은 비동기 제어의 네 가지를 스스로 결정하고 그 결정이 의도대로 동작하는지 검증한다. 마지막에 확정한 코드를 실물 장비에서 한 번 돌려 모형 결과와 비교한다. 평가는 결정과 검증을 본다.

1. 공통 안내

- 평가일: 2026 년 11 월 11 일(수), 1~8 교시. 제출 마감은 8 교시 종료.
- 평가 교과:「Project ④ 비동기 프로그래밍을 통한 실시간 PLC 제어」.
- 평가 도구: 포트폴리오와 평가자 체크리스트. 평가자가 제출 코드를 직접 실행해 학생의 예측과 대조한다. 100 점, 60 점 이상 PASS, 60 점 미만 FAIL.
- 제출은 LMS 에 ZIP 하나를 한 번 올린다. 점수와 개별 피드백도 LMS 에서 관리한다.
- AI 사용은 허용한다. 단, 1 교시 첫 판단 메모(40 분)와 8 교시 예측지(30 분)는 AI 와 코드 실행 없이 종이에 손으로 쓴다.
- 실습 장비는 1 대다. 구현과 검증은 교육 모형(래더 모형과 고장 재현)에서 하고, 7 교시에 학생마다 확정한 코드로 실물에서 반복 2 회를 한 번 실행한다(강사 입회). 모형 결과는 실물 결과가 아니다.

2. 학생용 과제

화요일 완성본에 반복 운전을 추가하라. 반복 횟수 N(1~5)을 받아 한 사이클을 N번 수행하고, 아래 고정 규칙을 지키며, 결정 항목 D1~D4는 스스로 정해 결정서에 근거와 함께 남긴 뒤 그대로 동작하게 하라.

코드는 AI가 작성해도 된다. 결정은 학생이 한다. AI가 결정서와 다르게 구현했다면 그것을 찾아 고치는 것까지가 과제다.

2.1 출발 코드와 계약

- 출발 코드는 배포한 `평가코드` 폴더다(화요일 A~D 완성본에 반복 운전의 자리만 만든 것). 수정은 `picknplace_full/sequence.py` 에서 하고, 화면을 연결한다면 `app.py` 와 `web/`도 고친다. 통신·모형 부품(`link.py`, `model.py`, `status.py`, `transport.py`)과 평가 도구는 고치지 않는다. 평가자는 이 부품을 배포 원본으로 바꿔 실행한다.
- 기존 A~D 검사가 `_accept_cycle_until` 의 호출 모양과 `stop_requested·move_timeout·grip_timeout` 의 이름과 쓰임을 고정한다. 다른 방식(예: `asyncio.Event` 로 정지 전달, 사이클 단위 한도)을 고르면 이 이름과 모양을 유지한 채 그 위에 얹는다(예: `stop_requested` 를 Event 를 읽는 property 로 둔다).

- 입구는 `Sequencer.start_repeat(count)`다. 1~5 정수가 아니거나 시작할 수 없으면(시작 버튼이 꺼지는 조건이거나 이미 반복·자동 순서 중이면) 아무것도 보내지 않고 이유 한 줄을 돌려준다. 시작하면 반복을 작업으로 띄우고 곧바로 한 줄을 돌려준다.
- 상태 속성 `repeating`(반복이 끝나기 전까지 참), `repeat_total`(N), `repeat_done`(완료한 사이클 수)의 이름과 뜻을 바꾸지 않는다.
- 반복이 끝나면 `last_result` 는 완료 `k/N`, 정지 `k/N`, 중단 `k/N:` 이유 중 하나다.
- 시간 한도는 `Sequencer` 의 기본값이나 반복 코드 안에 둔다. 검사기와 장면 실행기는 `Sequencer` 를 기본값으로 만든다.
- 화면 연결(횟수 입력, 반복 시작 버튼)은 해도 되고 안 해도 된다. 채점하지 않는다.

2.2 고정 규칙

모두에게 같다. 검사기 `check_eval.py`가 대표 장면으로 확인한다. 검사기 통과가 규칙을 모두 지켰다는 보증은 아니다.

1. 완료 횟수는 그 사이클의 5 단계 열림 완료를 새 관측으로 확인했을 때만 1 늘린다.
2. N 회를 넘겨 1111 을 보내지 않는다. 반복이 도는 동안 반복이나 한 사이클이 하나 더 시작되지 않는다.
3. 매 사이클 1111 전에 시작 버튼의 준비 조건(로봇 정지, 그리퍼 열림·동작 아님, 복구 불필요)을 확인한다. 1111 을 포함해 거부·미확인·시간 초과가 나면 반복을 끝낸다. 다시 보내거나 재시도하지 않는다.
4. 정지 요청은 기존 경계에서 처리한다. 1111 접수 뒤(이동 1 전)와 그리퍼 닫힘 확인 뒤에는 그 자리에서 9994 로 멈춘다. 이동 2 를 보낸 뒤에는 그 사이클을 끝내고 멈춘다. 사이클 사이(앞 사이클의 완료 관측부터 다음 1111 을 보내기 전까지)에 들어온 정지는 다음 1111 을 보내지 않고 정지 `k/N` 으로 끝낸다. 1111 을 보낸 뒤에 들어온 정지는 1111 접수 뒤 경계에서 멈춘다.
5. 즉시 정지는 반복 중에도 언제나 보낼 수 있다. 시간 한도 안에 재개되면 현재 사이클과 남은 횟수를 이어 간다. 한도를 넘기면 규칙 3 대로 반복을 끝낸다. 즉시 정지로 멈춘 시간을 한도에 넣을지는 D3 에서 정한다.
6. 이벤트 루프를 막지 않는다(`time.sleep` 같은 동기 대기 금지). `close()` 뒤에는 아무것도 보내지 않는다. 기존 A~D 검사를 통과한다.

2.3 결정 항목

정답은 하나가 아니다. 각 항목마다 선택지를 둘 이상 비교하고, 고른 이유를 "어느 await에서 무엇이 끼어들거나 끊기는지"로 설명하라. 화면 버튼은 HTTP 스레드에서 들어와 같은 이벤트 루프의 별도 코루틴으로 실행된다(`app.py`의 `call`). 반복 작업과 버튼 처리는 await 지점마다 번갈아 돈다.

항목	정할 것
D1 작업의 수명	반복을 어떤 작업(Task) 구조로 돌릴지(반복 전체를 작업 하나로, 사이클마다 작업 하나 등), 누가 그 작업을 들고 있을지, '실행 중' 표시를 언제 세우고 내릴지, `close()` 때 그 작업을 어떻게 끝낼지
D2 정지 요청의 전달	[정지]를 처리하는 코루틴이 남긴 요청을 반복 작업이 무엇으로 받을지(정한 지점에서 플래그 확인, `asyncio.Event`, `Task.cancel()` 등. `Task.cancel()`이라면 규칙 4를 지키려고 무엇을 더 해야 하는지까지), 사이클 사이의 정지를 어느 시점까지 받아 줄지(준비 확인 전·후, 1111 등록 직전), 한 사이클이 끝날 때 정지 요청을 지울지 남길지
D3 시간 한도	한도를 어디에 걸지(대기 한 번마다, 사이클마다, 반복 전체), 즉시 정지로 멈춘 시간을 포함할지, 값을 얼마로 할지(자기 측정 근거). 7교시 실물 확인도 같은 한도로 돈다
D4 새 관측의 기준	통신 작업이 따로 읽어 오는 상태 중 무엇을 판단에 쓸지. 앞 사이클의 완료에 쓴 관측을 다음 사이클의 시작 판단에 다시 쓸지, 대기를 부르기 전에 이미 도착한 관측과 통신 실패 중에 남아 있는 마지막 값을 어떻게 다룰지

2.4 제공 도구

평가코드 폴더에서 실행한다.

명령	하는 일
`python -B -X utf8 check_eval.py`	고정 규칙 검사(모형 4배속)와 기존 A~D 검사
`python -B -X utf8 eval_scenarios.py normal --count 2`	반복 N회 실행. 단계별 시간 측정에 쓴다
`python -B -X utf8 eval_scenarios.py s1 --order repeat-first`	반복 시작과 개별 [스크립트1] 요청이 같은 순간 들어온다. `script-first`도 있다
`python -B -X utf8 eval_scenarios.py s2 --cycle 1 --delay 0.1`	사이클 완료를 PC가 관측한 뒤 지정한 초에 [정지]
`python -B -X utf8 eval_scenarios.py s3 --cycle 1 --move 1 --hold 20`	이동 중 [즉시 정지], 지정한 초 뒤 [재개]
`python -B -X utf8 eval_scenarios.py s4 --cycle 1 --drop 3`	사이클 완료를 PC가 관측한 직후 통신이 지정한 초 동안 끊김
`python -B -X utf8 eval_harness.py`	지금 코드의 지문(12자리)

명령	하는 일
`python -B -X utf8 eval_real.py --target tcp:127.0.0.1:5020 --count 2`	실물 확인 리허설. 가짜 PLC(`python -B -X utf8 -m fake_plc`)와 가짜 로봇(`python -B -X utf8 -m fake_robot`)을 먼저 띄운다
`python -B -X utf8 eval_real.py --target auto --count 2`	7교시 실물 확인(강사 입회). 장비에 연결한 자기 노트북에서 실행한다

장면 실행기는 1배속 모형으로 돌고 `logs/eval_<장면>_<시각>.log`를 남긴다. 실물 실행기는 `logs/eval_real_<시각>.log`를 남긴다. 로그 맨 위에는 코드 지문이 찍힌다. normal 장면과 실물 실행기는 끝에 같은 기준의 단계별 시간 표(이동1, 닫힘, 이동2, 열림)를 찍으니, 둘을 나란히 놓고 비교한다. 실물 실행기는 연결 뒤 3.5초 상태를 지켜본 다음 시작 조건을 보고, 조건이 아니면 아무것도 보내지 않는다. S1~S4는 8교시 예측지의 장면과 종류가 같다. 8교시에는 값(횟수, 순서, 사이클, 초)을 알려 준다. 위 예시와 같은 값일 수도, 다른 값일 수도 있다.

2.5 진행

시간	할 일
1교시	과제 확인 뒤 첫 판단 메모(40분, AI·실행 없이 손글씨). D1~D4 각각 처음 고른 것과 이유. 다 쓰면 바로 사진을 찍고 원본은 강사에게 낸다. 바뀐 판단은 결정서에 적는다
2~6교시	결정서를 AI에게 주고 구현시킨다. 검사기와 장면 실행기로 확인하고, 필요하면 결정을 바꿔 다시 구현·검증한다. 실물 확인 리허설(가짜 PLC·가짜 로봇)을 한 번 해 둔다
6교시 끝	코드를 확정한다. `python -B -X utf8 eval_harness.py`의 지문을 강사 명단에 적는다(제출 아님). 이후 `picknplace_full/`을 고치지 않는다
7교시	실물 확인(1인 약 4분, 강사 입회, 명단 순서). 장비에 연결한 자기 노트북에서 `eval_real.py --target auto --count 2`를 실행한다. 차례를 기다리는 동안 확정한 코드로 최소 장면을 다시 실행해 검증 기록을 마무리하고, 실물 결과를 모형 normal 결과와 비교해 적는다
8교시	예측지(30분, 손글씨). 자기 코드와 결정서는 볼 수 있다. 로그·검증 기록·AI·실행은 쓰지 않는다. 맨 위에 코드 지문을 적고, 강사가 알려 준 S1~S4 값에서 자기 프로그램이 보낼 명령과 판정, 반복 결과, 근거 결정을 쓴다. 그다음 ZIP을 제출한다

3. 제출물

ZIP 하나를 LMS에 올린다.

- `picknplace_full/` 폴더 전체(확정한 코드).
- 결정서: D1~D4 마다 첫 판단, 비교한 선택지, 최종 선택, 이유, 근거 로그, 동작 진술(조건 → 결과), 구현 위치(파일:줄), 유지·변경과 그 계기. 배포한 양식을 쓴다.
- 검증 기록: 제공 장면의 실행 전 예측, 실제 결과, 일치 여부, 불일치 원인과 조치. 최소 장면은 normal 1 회, S1 두 순서, S2·S3·S4 각각 값 두 가지로 모두 9 개다. 최소 장면의 로그는 확정한 코드로 돌린 것이어야 한다(로그 맨 위 지문이 제출 코드와 같음). 결정을 바꾼 계기로 붙이는 이전 판 로그는 "이전 판"이라고 표시한다. 해당 `logs/eval_*.log` 와 `check_eval.py` 출력을 함께 넣는다.
- 실물 확인: `logs/eval_real_*.log` 와 비교 메모. 모형 normal 과 실물의 단계별 시간과 결과를 나란히 적고, 자기 한도(D3)와 관측 기준(D4)이 실물에서도 맞았는지, 다르면 왜 다른지 쓴다.
- 첫 판단 메모 사진과 예측지 사진.

결정서, 검증 기록, 실물 비교는 한 문서에 절을 나눠도 된다. AI와 나눈 대화 전체는 제출하지 않는다.

4. 채점표

각 요소는 0점·절반·만점으로 채점한다. 증거가 없거나 절반 기준에도 미달하면 0점이다.

항목	요소	배점	만점	절반
R1	결정 D1 작업의 수명	6	다섯 가지를 모두 갖춤: ① 선택지 둘 이상 비교 ② 이유를 어느 await에서 무엇이 끼어들거나 끊기는지로 설명 ③ 근거 장면 로그 ④ 확인할 수 있는 동작 진술 ⑤ 구현 위치	다섯 중 셋 이상을 갖춤
R2	결정 D2 정지 요청의 전달	6	R1과 같음	R1과 같음
R3	결정 D3 시간 한도	6	R1과 같음. 값이 자기 측정에 근거함	R1과 같음. 측정 근거가 없으면 최대 절반
R4	결정 D4 새 관측의 기준	6	R1과 같음	R1과 같음

항목	요소	배점	만점	절반
R5	첫 판단과 변경	6	첫 판단 메모에 D1~D4의 선택과 이유가 모두 있고, 이유가 비동기 동작(대기, 끼어듦, 관측 시점)으로 쓰임. 결정서가 항목마다 유지·변경과 그 계기가 된 장면을 적음	메모 일부가 비었거나 이유가 비동기 동작과 무관함. 또는 변경 이유가 실험과 연결되지 않음
R6	검증 기록	10	최소 장면 9개를 모두 실행 전 예측, 실제, 일치, 불일치 원인과 조치로 기록함. 최소 장면 로그의 지문이 제출 코드와 같음(표시한 이전 판 로그는 감점하지 않음)	최소 장면 절반 이상만 기록했거나, 최소 장면 로그 일부가 제출 코드와 다른 판임
R7	고정 규칙 검사기	10	평가자가 제출 코드로 실행한 검사 15개(규칙 검사 14개와 기존 A~D 검사)를 모두 통과함	15개 중 8개 이상 통과함
R8	예측 대조 S1	10	(가) 명령과 판정의 순서와 (나) 반복 결과(종류와 k/N)가 평가자 실행과 일치하고, (다) 근거가 결정서의 해당 결정으로 설명됨	(가)(나)가 모두 맞으나 (다)가 결정과 연결되지 않음. 또는 (다)가 결정으로 설명되고 (가)(나) 중 하나만 맞음. 또는 예측은 결정서대로이나 구현이 결정서와 달라 결과가 다름
R9	예측 대조 S2	10	R8과 같음	R8과 같음
R10	예측 대조 S3	10	R8과 같음	R8과 같음
R11	예측 대조 S4	10	R8과 같음	R8과 같음
R12	실물 확인	10	강사 입회 아래 확정한 코드로 실물 반복 2회가 완료 2/2로 끝남. 실물 로그와 모형 normal 로그의 단계별 시간·결과를 비교해 자기 한도(D3)와 관측 기준(D4)이 실물에서도 맞는지 설명함	완료하지 못했으나 그 원인을 실물 로그와 자기 결정으로 설명함. 또는 완료했으나 비교 설명이 없음

R1~R6은 결정서·첫 판단 메모·검증 기록, R7은 평가자의 검사기 실행, R8~R11은 예측지와 평가자의 장면 실행, R12는 강사 입회 기록(명단)과 실물 로그·비교 메모가 근거다.

- 실물 확인: 실물 로그의 지문은 명단의 지문과 같아야 한다. 장비 고장처럼 학생 책임이 아닌 이유로 실물을 돌리지 못하면, 가짜 PLC·가짜 로봇 리허설 로그(`eval_real.py --target tcp:…`)로 R12 를 대신 채점한다. 자기 차례에 코드가 준비되지 않아 실행하지 못했으면 R12 는 0 점이다.
- 코드 지문: 6 교시 끝 명단의 지문과 제출 코드의 지문이 다르면(확정 뒤 수정) R8~R12 는 0 점이다. 예측지에 적은 지문만 한두 글자 다르고 명단·제출 코드의 지문이 같으면 옮겨 적기 실수로 보고 인정한다.
- 반복을 구현하지 않아 어느 장면에서도 1111 이 나가지 않는 제출(출발 코드 그대로 등)은 R8~R11 을 0 점으로 한다.
- 실행 실패로 판단할 수 없는 다른 요소까지 일괄 0 점 처리하지 않는다.
- 첫 판단이 틀렸다고 감점하지 않고, 바꿨다고 가점하지도 않는다. 처음 판단이 타당했고 검증도 통과했다면 그대로 인정한다.
- AI 사용 여부, 모델 종류, 사용량으로 가감점하지 않는다. 문서 분량은 점수 기준이 아니다.
- 실패한 실행도 관찰 근거가 있으면 검증으로 인정한다.